

Build the Smallest ESP32 Brushless Rocket Drone | ESP-BLAST

By Max Imagination in Circuits > Remote Control  35,908  212  5 ★ Featured



Build The Smallest ESP32 Brushless Rocket Drone | ESP-BLAST

Max Imagination



Watch on



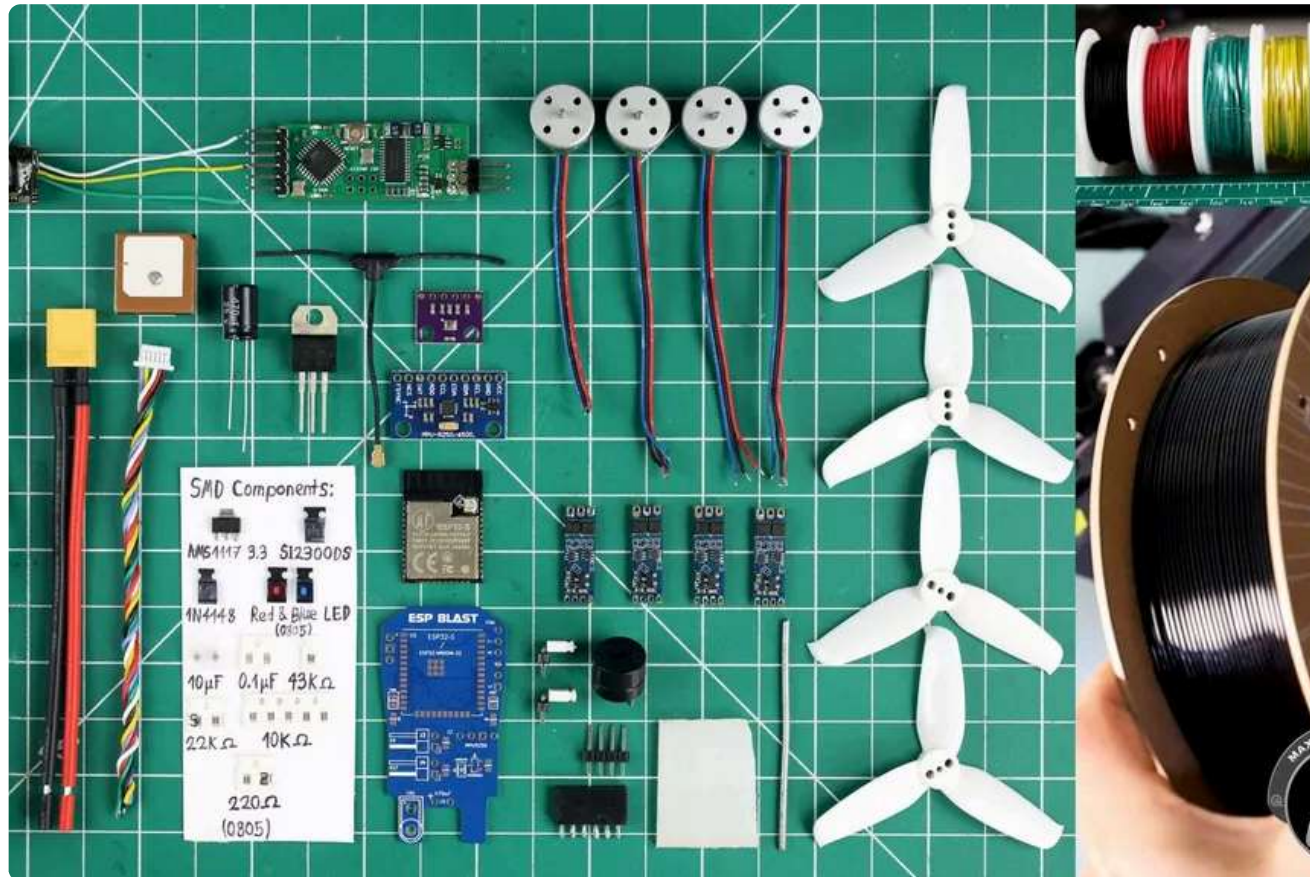
This is the ESP-BLAST - the smallest rocket drone you can build for insanely fun, high-speed flights exceeding 100km/h.

Inspired by South African and Australian builders pushing record-breaking speeds with their larger drones, this little one packs an ESP32 at its core inside an aerodynamic bullet frame, driven by four tiny brushless motors that make it rip through the air. It even comes equipped with a motorized

FPV camera that tilts to keep your view locked in as you transition from vertical to horizontal flight.

I'm Max, and in this tutorial, you're in for a "blast" as I deliver the files, bill of materials, and full instructions to go from concept to a working high-speed mini quad built purely for fun!

Supplies



Let's take a closer look at the materials and parts which include a set of 4 micro ESC's driving 1104 brushless motors that move two and a half inch props, through hole parts and surface mount components that go onto a

custom PCB, an ESP32 module with antenna, barometer, IMU, micro FPV camera with a module for on-screen-display, GPS, capacitor, regulator, a 3S LiPo with an XT30 connector for power, 30-gauge silicone wire, and PETG filament for the frame.

All up, you're looking at roughly \$155 in parts actually put into the drone as of March 2026 excluding the cost of spare parts, shipping, and the 3D design.

Find the full materials list with product links in the pinned comment below the video tutorial: <https://www.youtube.com/watch?v=pUi1T12QYAU&t=3s>

3D model/design of ESP-BLAST's Parts (STLs, STEP, & G-CODE):
<https://cults3d.com/en/3d-model/game/esp-blast-an-esp32-mini-rocket-drone-100km-h-3d-design-files>

Remaining Project Files - (PCB Files and Diagrams):
https://drive.google.com/drive/folders/1vqXdgHj-5dydJ4V21HF_LdW4IPgcUf8g?usp=sharing

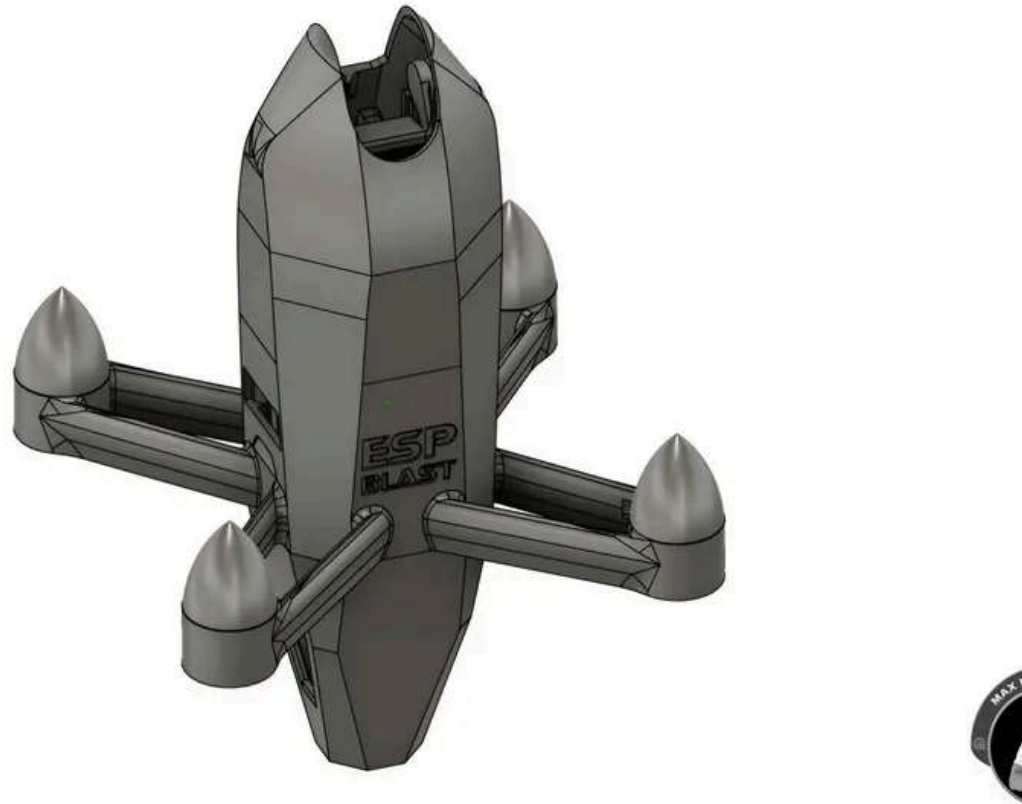
ESP-FC Firmware (Thanks to rtlopez) - <https://github.com/rtlopez/esp-fc/tree/master>

Radio Controller & ESP-FC Firmware Instructions - <https://youtu.be/QTmitUFotik>

MWOSD Firmware - <https://github.com/ShikOfTheRa/scarab-osd>

Learn more about MinimOSD - <https://ardupilot.org/copter/docs/common-minim-osd-quick-installation-guide.html>

Step 1: The 3D Design



I came up with the 3D design by taking measurements of every single component that would go into the rocket drone and 3D modeled them into Fusion.

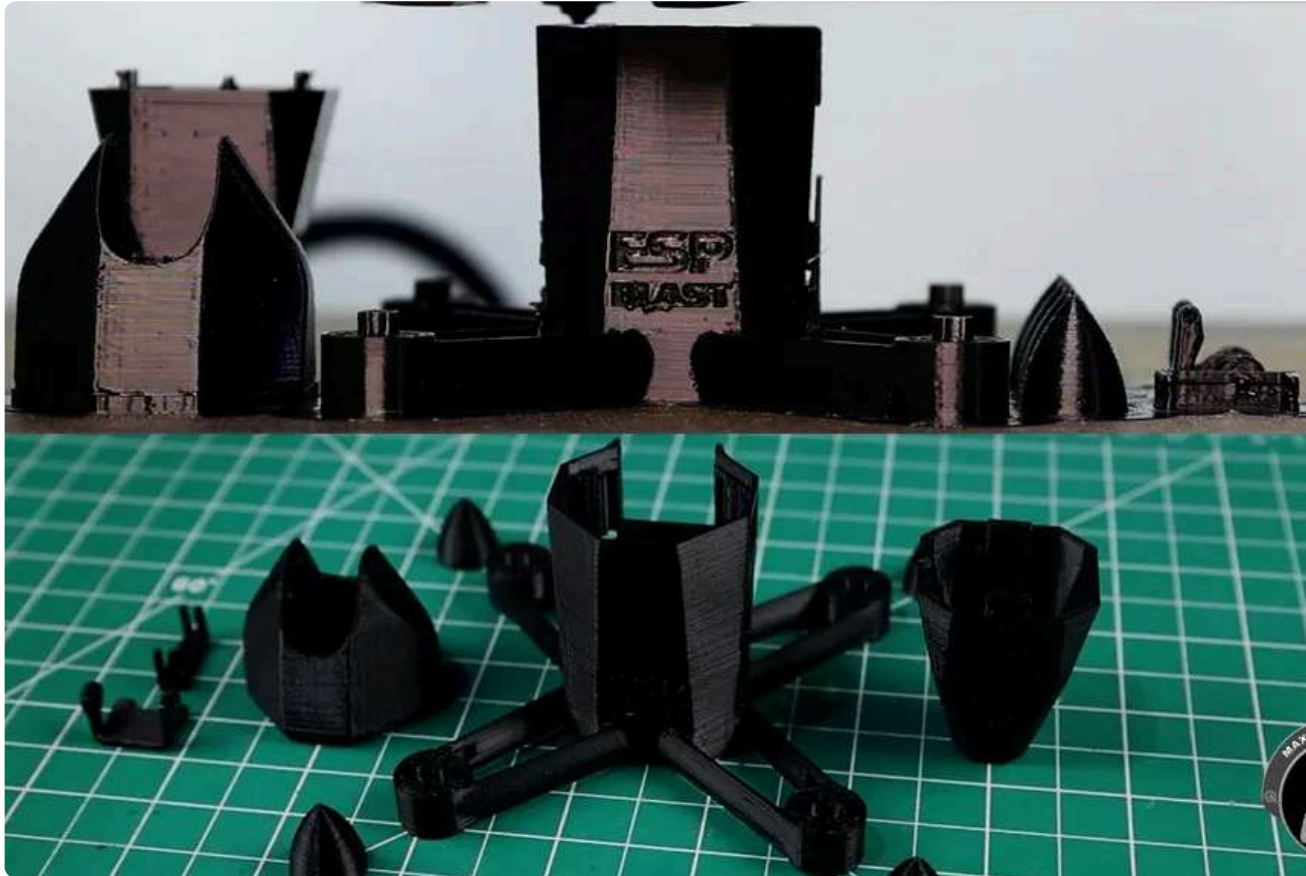
I positioned these parts around the battery and outlined a PCB to hold some of them, and then modeled the frame which contains most parts, the nose cone that houses the tilting FPV camera with its mechanism, and the tail cone as a lid for where the battery slides in.

I worked further on openings for a GPS, LEDs, and barometer as well as mounts for internal modules like the ESCs, MinimOSD, and flight controller.

Finished off with motor cones and improved arms containing channels for routing motor wires and a more aerodynamic look, completing the mini rocket drone design which I put through AirShaper to confirm it is aerodynamic enough at high speeds.

To get the 3D design, head over to Cults3D: <https://cults3d.com/en/3d-model/game/esp-blast-an-esp32-mini-rocket-drone-100km-h-3d-design-files>

Step 2: 3D Printing the Rocket Drone's Parts



Next, we'll drop the STLs into the slicer, position each part on its flattest side, and use the PETG settings I've found work best - which you can

copy, along with the ready-to-go g-code, from the project files folder linked above.

Let's send it to the printer and heat her up. For filament, I'm using the durable PETG from Elegoo. PLA would snap on impact and soften in the sun where PETG handles both much better. If you live in a humid climate, printing from a dry box is non-negotiable.

Let's say you're not using my g-code, then lower the frame on the z-axis by 0.2 mm to ensure the brim wraps fully around instead of letting the arms lift during printing.

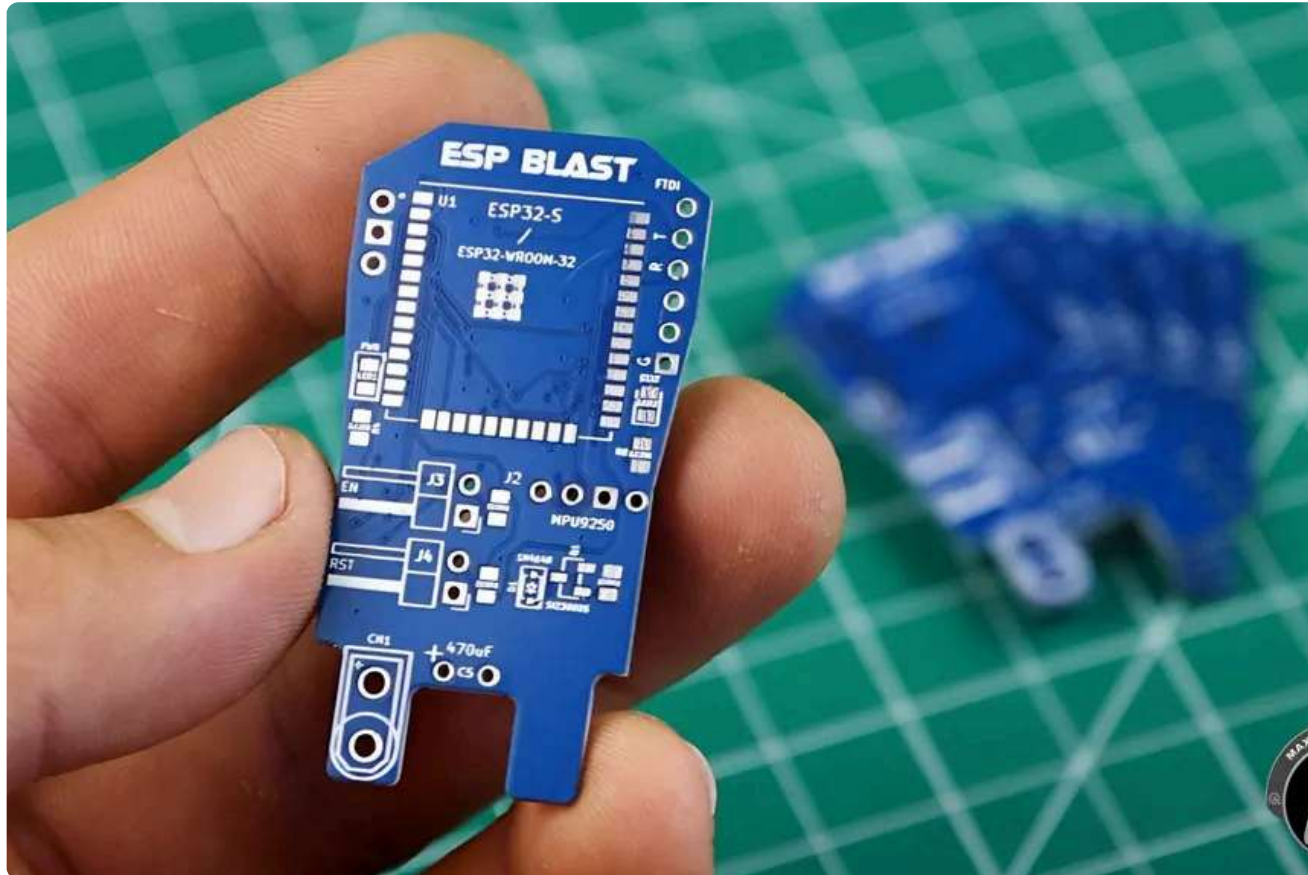
Even though my printer can reach 500 mm/s, I slowed it to 80 as PETG doesn't like being rushed. That's also why I chose a 0.2 mm layer height; slightly taller layers help reduce PETG's tendency to ooze and preserve surface quality.

Those settings worked well for me, but I'm always open to improvements from you 3D printing pros.

After about two and a half hours, we can pop the parts off, remove the brims and supports, and reveal the final result - 40 grams of lightweight,

durable PETG frame and body parts ready for the rocket drone.

Step 3: Ordering the Flight Controller's PCB



Now we need to get the PCB made, and let me show how you can get it too

To get yours; first, download my custom PCB design's gerber file linked with the project files: https://drive.google.com/drive/folders/1vqXdgHj-5dydJ4V21HF_LdW4IPgcUf8g?usp=sharing

Then, head to JLCPCB for an instant quote where you upload the gerber file and select its board specs:

- Keep it at **4 layers**.
- Choose how many PCBs you want (minimum 5)
- Set the **board thickness** to 1.2 mm.
- Pick your favorite board color.

To make soldering SMD components way easier, I always add a PCB stencil. Select both sides, set a custom size of 70 by 80 mm, choose your quantity, and you're good to go.

From there, just add those to your cart, check out, and give JLC about 48 hours to manufacture it. A few days later, you'll get a cool blue box at your door with the PCBs and stencil ready for soldering.

I'm always impressed with the quality and consistency of JLCPCB's boards.

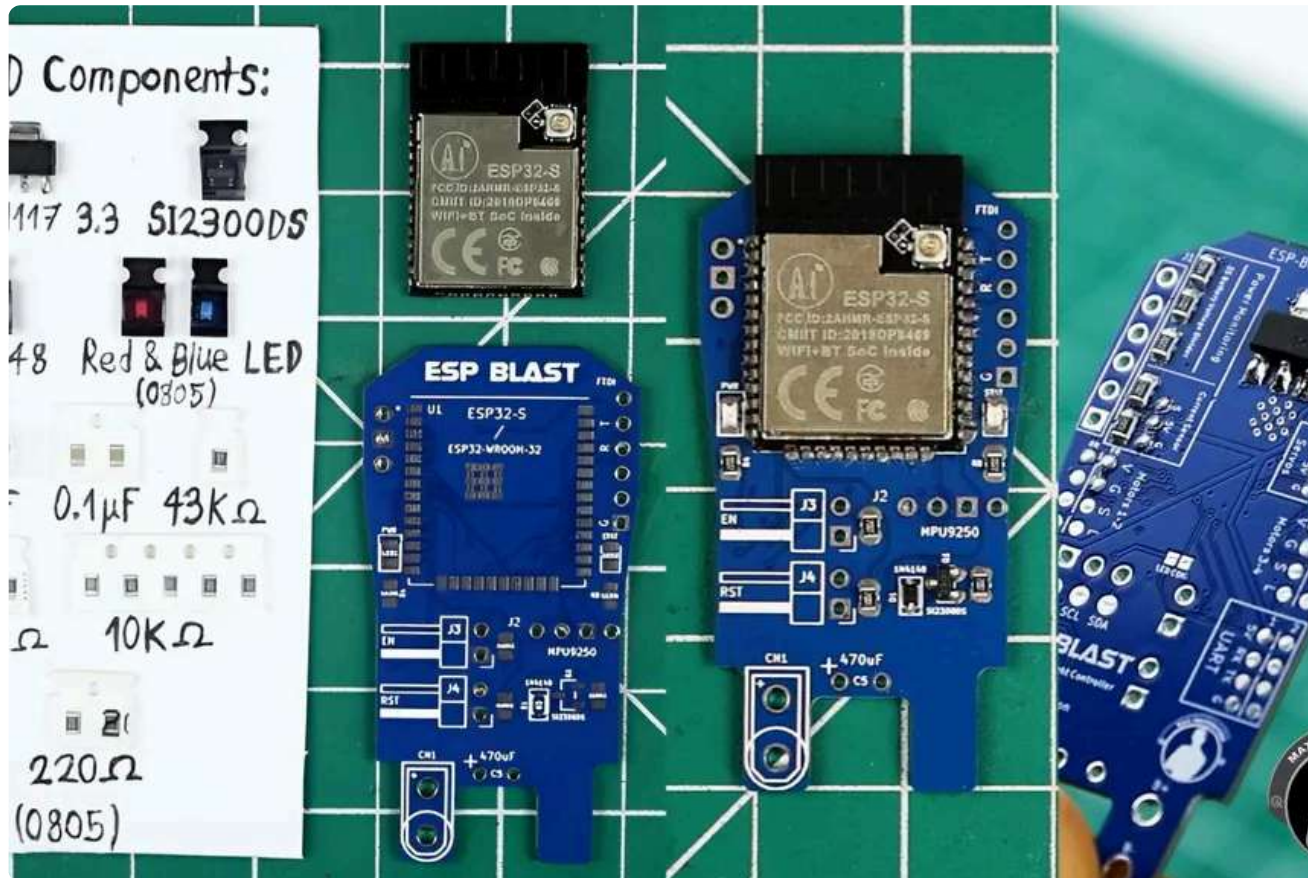
And fun fact, in their most affordable color green, you can get these for just \$7.

Plus, for smaller boards, their 2 to 8-layer designs start as low as \$2.

If you're new, signing up gets you \$123 in coupons to kickstart your first order.

Order it here: <https://jlcpcb.com/?from=MaxImagination>

Step 4: Installing SMD Components



To assemble the PCB, you can either have JLC build it for you using the additional manufacturing files I've provided - like I've shown in previous tutorials, or assemble it yourself, which is exactly what we're about to do.

Starting with SMD components:

Positioning one PCB with another 4 around it keeps it from moving around while we apply solder paste onto the top side pads with the stencil we ordered. Just keep it pressed down while spreading paste over with the scraper on a low angle and scrape off excess paste holding it on a steeper angle, then remove the stencil.

Now for manual pick and place:

We'll drop the MOSFET for controlling the alarm along with its flyback diode and pull-down resistor with others for pulling up reset and enable pins, LEDs for power and ready status indication with their own current limiting resistors, and then the ESP32 module.

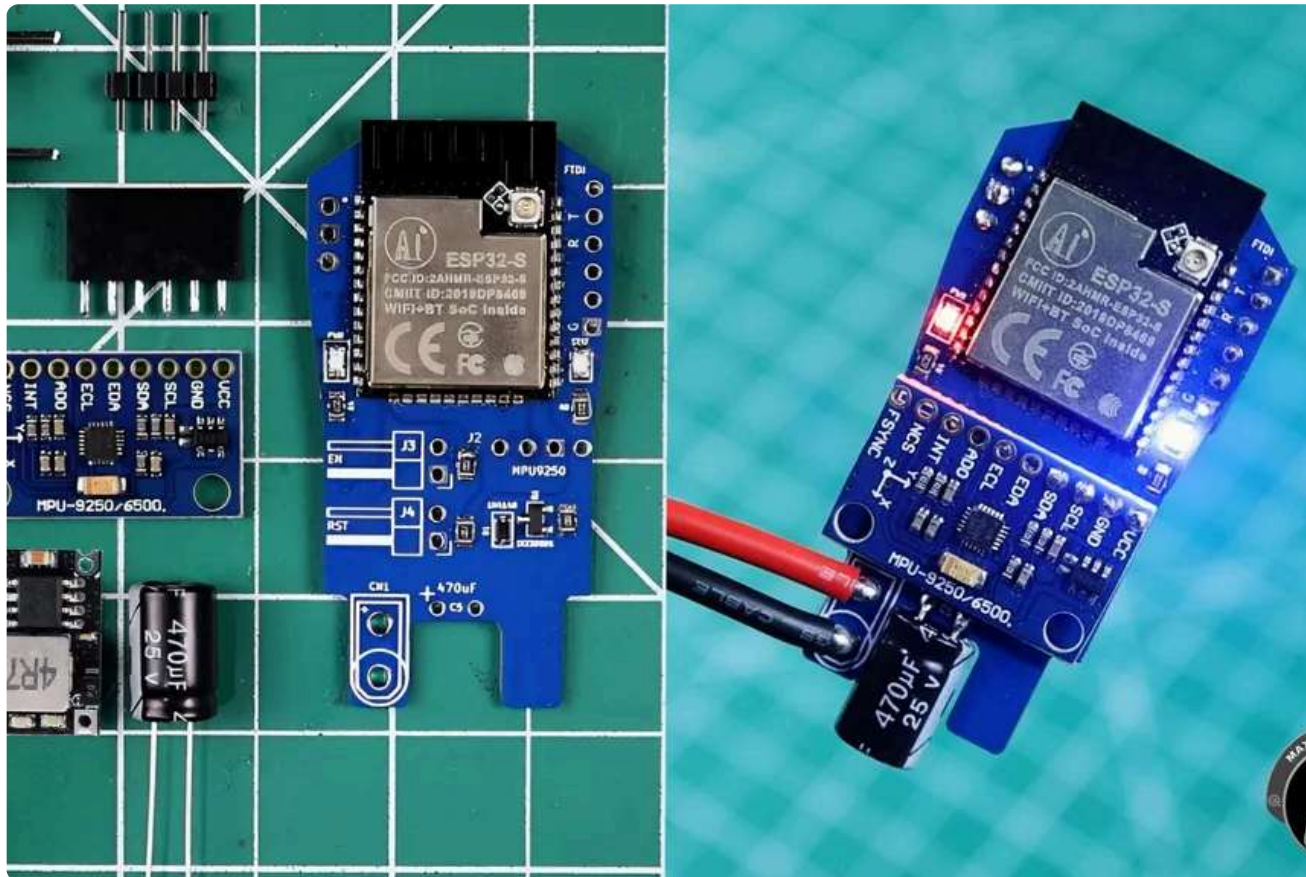
Let's heat up our hot plate to 200 degrees celsius and place the board for a brief moment until all solder joints have formed.

Now, let's slap on the other side of the stencil onto the back of the flight controller with some electrical tape - make sure it doesn't move as you're applying paste.

Here, we'll place the 3.3V regulator and its power filtering capacitors, voltage-dividing resistors so the ESP32 can read the battery voltage and measure current if you decide to add that module (unlike I did).

Then, we'll use a hot air gun to get them wiggling into their spots. You can alternatively use a soldering iron for this entire board - it will only take you longer, but will work fine.

Step 5: Installing Through-hole Parts



With the SMD show over, time for THT parts; starting by inserting male and female pin headers.

Adding a dot of superglue helps hold them in place as you solder the pins on the other side.

Trim any excess pins flat, especially on the programming header as it will rub up against the frame.

Next, we'll install a 5V DC buck converter to step-down the battery's 12V to 5V efficiently.

At the bottom, goes a 470uF electrolytic capacitor for buffering voltage ripples - super important in FPV drones. Keep in mind its polarity as you insert it.

The flight controller is nearly assembled, we just need to solder on an XT30 power connector for the battery to plug into and install the MPU9250 9-axis Inertial Measurement Unit (IMU) which doesn't quite match the width of the main board, so we therefore need to sand down the sides enough for it to fit.

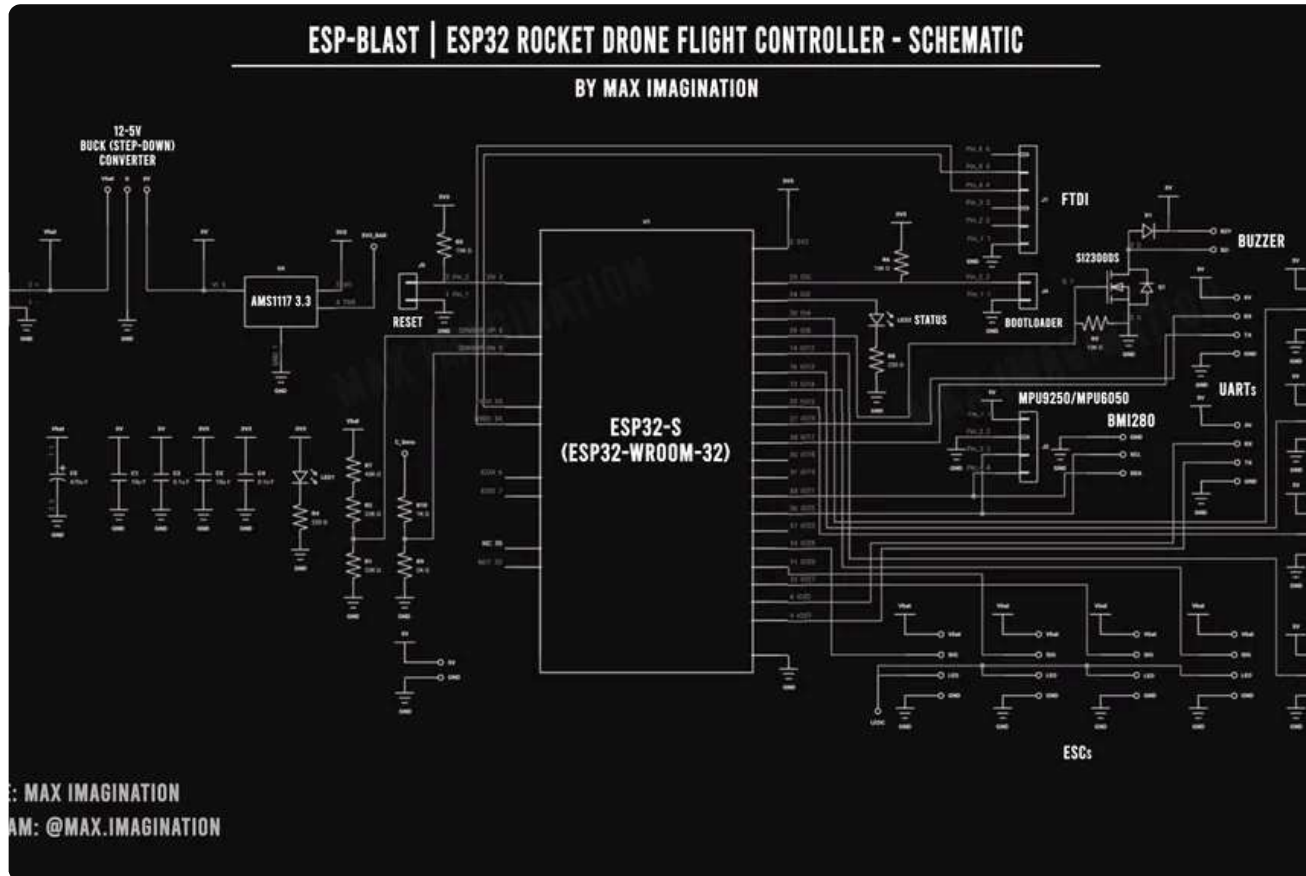
This 3-in-1 module features a built-in gyro, accelerometer, and magnetometer for orientation, motion-tracking, and navigation. The latter

two features won't be seen used in this video, but will be reserved for the possibility of future autonomous flight - giving it more potential.

The flight controller is complete and powers up on a 3-cell battery - but before we move on, we need to desolder this 0-ohm resistor from the ESP32 module. It currently routes WiFi signal to the onboard PCB antenna, and we'll replace it with a solder bridge to the u.FL connector, so we can use an external antenna and increase range.

If you have the ESP32-WROOM-32 version of this module, you might be stuck with the PCB antenna, but that can still give some decent range.

Step 6: PCB Schematic



If you need to verify internal FC connections or design your own Flight Controller for a different drone, refer to the ESP-BLAST's PCB schematic.

Step 7: Wiring the Test Circuit



To ensure the whole circuit works before shoving it into the frame, let's wire the remaining modules to the flight controller in the most basic, direct way possible.

Starting with the “JMT HLK-DL03” 8A micro brushless ESCs - which we’ll pre-wire with 30-gauge silicone wire. You can use thicker wire for power and ground or simply twist two together, and use individual wires for signal in yellow and LED in green.

Then, we’ll connect up the motors in opposing polarities per pair - repeat these steps for the other pair of ESCs.

We’ll pre-wire the BMP280 barometer with lines for I2C and power, expose a servo’s wires, and give the buzzer a couple.

For the MinimOSD module - which is split into a digital side with the microcontroller and the analog side with the OSD chip, we need to keep the power pads on the top of the board bridged, which by default should already come adjoined via a thin PCB trace - so both get powered with one voltage supply line. If these don’t come bridged, connecting them with a blob of solder will do the trick.

We’ll solder the camera’s wires for power and ground to the minimOSD, then extend its power line to 5V. Make sure that the ground line on both sides of the minimOSD module is common. On the camera, the green wire

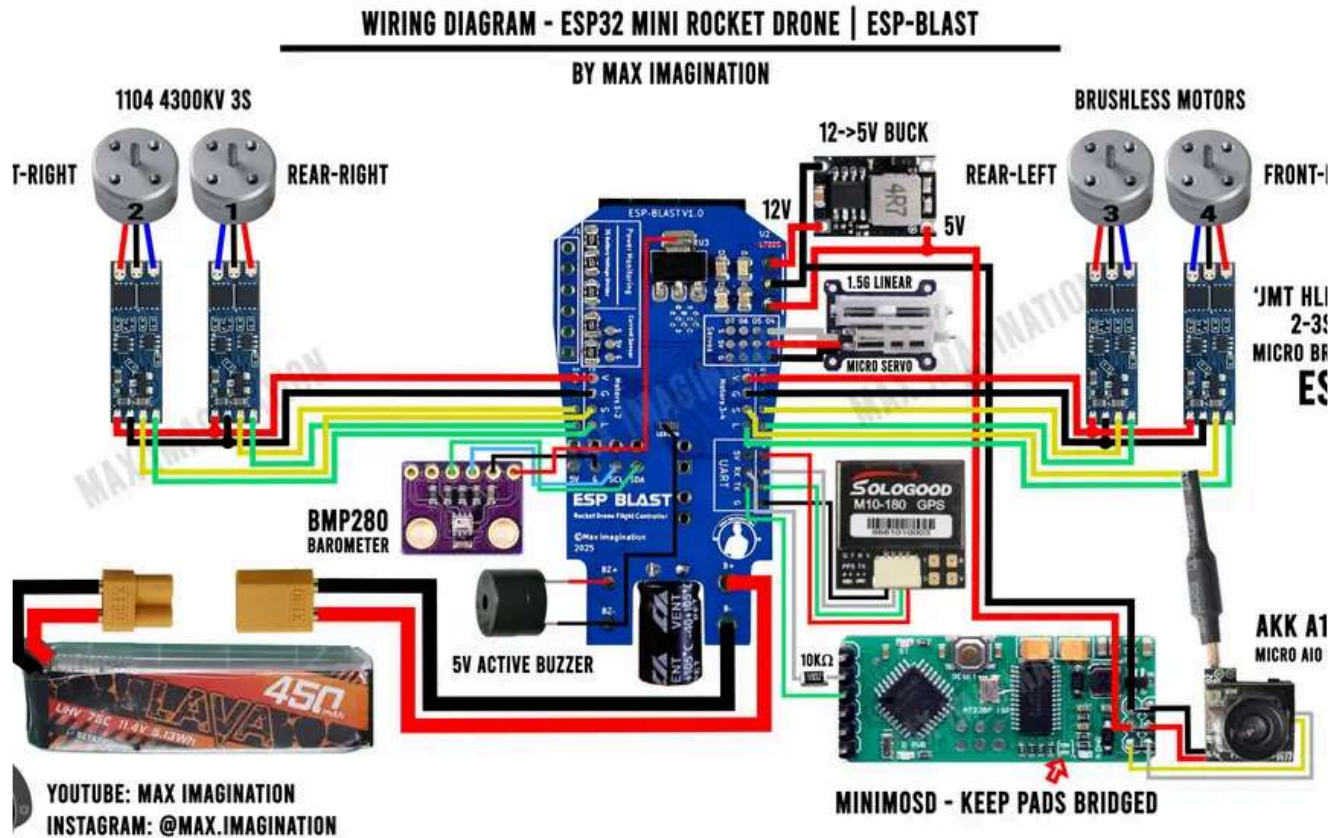
is for Smartaudio control which won't be used, therefore, only solder its remaining video in and video out wires (VIN & VOUT) to the minimOSD.

Let's finish prewiring by soldering a couple of TX/RX UART lines for the digital side with a 10K-ohm resistor at the TX side only to drop logic voltage to a safe level that can be read by the ESP32. Using a resistor is the most convenient way to adapt for the right logic voltage instead of using a logic converter.

With all the modules pre-wired, we can start at the back of the flight controller, prop it up with a set of helping hands and begin by hooking up the ESCs powered directly from the battery, the 5V micro servo for tilting the camera, and barometer for altitude data powered via 3.3V.

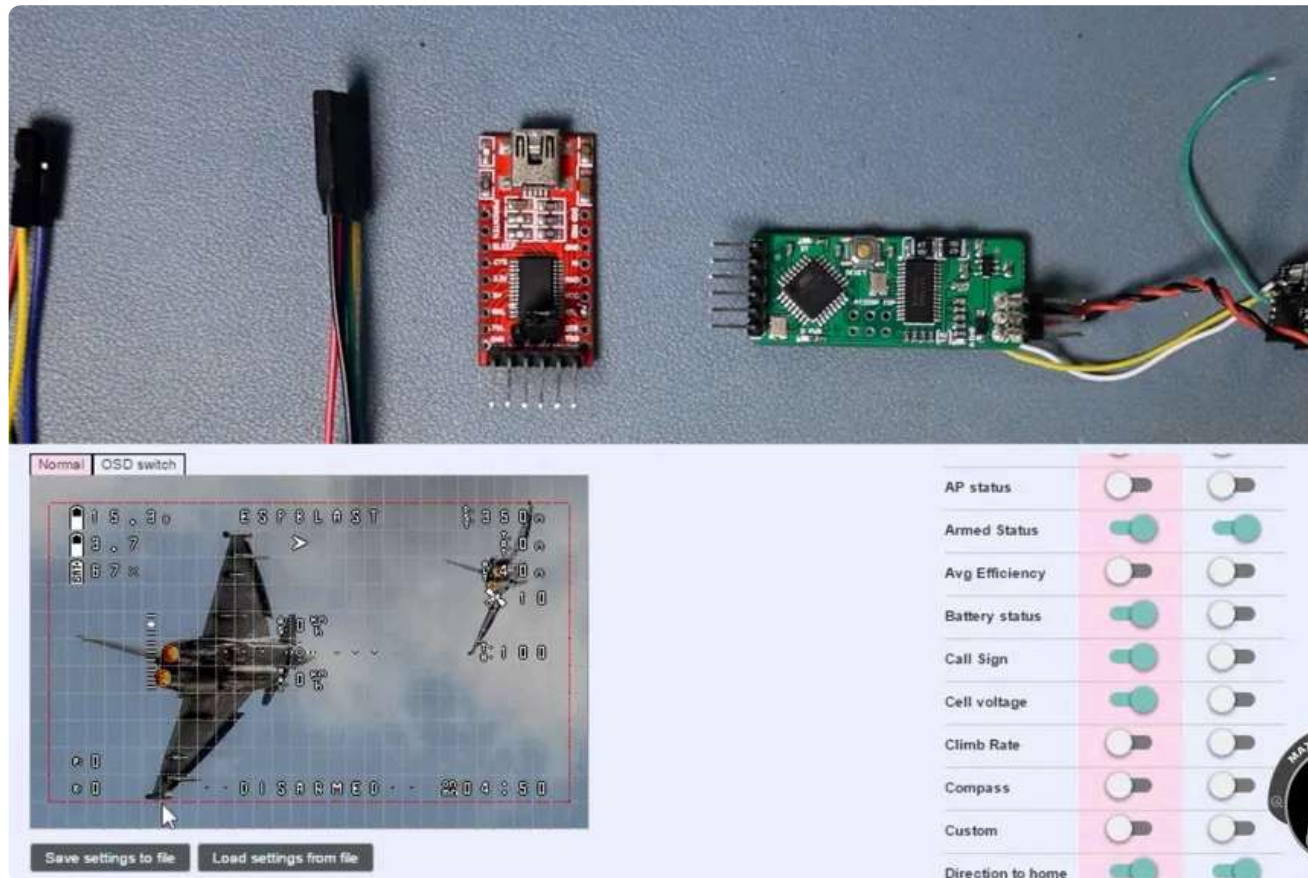
Then, goes the alarm buzzer, the GPS for speed readings, and last, but not least: the minimOSD module powered from 5V at the camera side.

Step 8: Wiring Diagram



Refer to this easy, complete wiring diagram as you're soldering wires for both the test circuit and final wire-up inside the drone.

Step 9: Programming MinimOSD



Now that the test circuit wiring is complete, we'll power it all up.

On my end, it might already be running the pre-programmed firmware and On-Screen-Display seen in my headset, but let's go through the

programming together; starting with the minimOSD module and camera on their own.

We'll need some cable and what's called an FTDI - USB to Serial converter for programming. Use a set of 5 Female-to-female jumper wires connecting to Ground, VCC, TX, RX, and DTR pins. Then, plug the other ends into the minimOSD, making sure TX goes to RX and vice versa, connect the converter to your computer while ensuring its power jumper sits on the 5V side.

We'll head to the computer and go to the repository for the minimOSD's firmware known as MultiWiiOSD and download the zip:

<https://github.com/ShikOfTheRa/scarab-osd>

Then, we'll jump into the folders and open up the MultiWiiOSD configurator.

At the top right, we'll select the COM port and before hitting connect, we need to click on Flash where a new window appears. Select the following Version, Device, Variant, and Type from the drop-downs. Load the online hex firmware and then hit flash firmware. Whatever you do, do not bump the modules while the firmware is being flashed.

Once done, hit connect, it should give you a waiting message followed by “Read Done” showing your minimOSD is communicating.

In the tab under Main Voltage, select the voltage source, make it use the battery cell count, voltage alarm to go at 3.4V, a cell count of 3, and a voltage adjust value of 200. In the Amperage tab, no need to set anything here unless you have a current sensor connected. In the other tabs, keep the selections that are there by default or play around with them to your liking as long as you know what you’re doing.

In the OSD selector panel is where you do the customization of adding or removing readings such as:

- Speed
- Maximum speed
- Distance from home
- Total distance traveled
- GPS altitude
- Barometer altitude
- Satellite count
- Compass

- Direction to home arrow
- Artificial horizon indicators
- Throttle percentage
- Cell voltage and total voltage
- Flight timer
- Arming status

And many more...

Once you have the info you want displayed, organize the position of each icon to your liking and even add your aircraft name as the callsign.

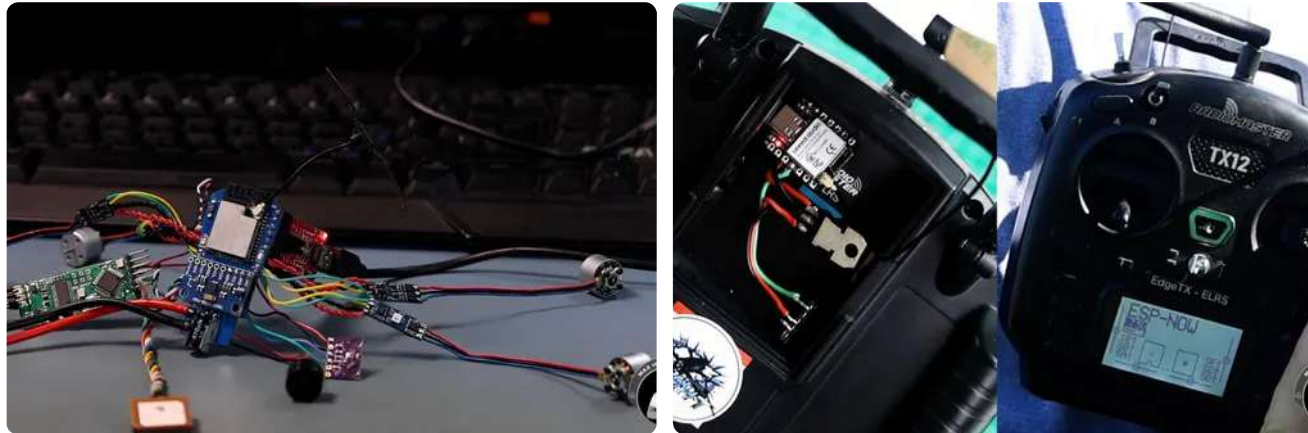
By the way, I have saved this OSD layout as a file to load onto your minimOSD - simply find it among the project files:

https://drive.google.com/drive/folders/1vqXdgHj-5dydJ4V21HF_LdW4IPgcUf8g?usp=sharing

To save the changes, click *Write* - the minimOSD will store the layout.

Then, we can disconnect it from the computer and wire it back to the flight controller.

Step 10: Programming Flight Controller and Setting Up TX



Before testing the minimOSD and camera, we'll need to connect the flight controller to the computer and flash its firmware.

We use the same FTDI, with only RX, TX, and GND at play which means it still needs the battery connected. Before doing so, we need to bridge the RST pins behind the MPU9250 with a jumper, plug the FTDI into the computer, and plug in the battery.

Once powered on, briefly short the EN pins above the RST jumper to put the ESP32 into bootloader mode.

Then, head over to the *esp-fc* repository by rtlopez:

<https://github.com/rtlopez/esp-fc/tree/master>

It is an ESP32-based flight controller firmware I covered in detail in my last video: <https://youtu.be/QTmitUFotik>

To flash the firmware, we will NOT be using the binary file and ESP Tool site to flash it as we are using files from the up-to-date master branch of this repository.

Download the zip file, open Visual Studio Code with the PlatformIO extension installed, ensure the folder is opened in the explorer tab, and open the platformio.ini file.

Since we'll be flashing for the ESP32 environment, type the line: *pio run -e esp32 -t upload* to both compile the firmware and flash it to the chip.

Once successful, power off and unplug the flight controller, remove the jumper that we placed on the reset header and reconnect the flight controller.

To configure the flight controller, let's head into Betaflight 10.10, select the COM port, hit connect, ignore the warnings, and open the command line

interface (CLI).

Typing *get pin* shows the default pin assignments for each function.

However, to match our hardware, we'll use this custom pin table defining the motors, servo, buzzer, LED, UART ports for the GPS and minimOSD, the I2C lines for the IMU, and the ADC pin for voltage sensing:

Command: get pin

```
set pin_input_rx -1
```

```
set pin_output_0 25
```

```
set pin_output_1 26
```

```
set pin_output_2 14
```

```
set pin_output_3 27
```

```
set pin_output_4 12
```

```
set pin_output_5 -1
```

```
set pin_output_6 -1
```

```
set pin_output_7 -1
```

```
set pin_button -1
```

```
set pin_buzzer 5
```

```
set pin_led 2
```

```
set pin_serial_0_tx 1
```

```
set pin_serial_0_rx 3
```

```
set pin_serial_1_tx 33
```

```
set pin_serial_1_rx 32
```

```
set pin_serial_2_tx 17
```

```
set pin_serial_2_rx 16
```

```
set pin_i2c_scl 22
```

```
set pin_i2c_sda 21
```

```
set pin_input_adc_0 36
```

```
set pin_input_adc_1 -1
```

```
set pin_spi_0_sck -1
```

```
set pin_spi_0_mosi -1
```

```
set pin_spi_0_miso -1
```

```
set pin_spi_cs_0 -1
```

```
set pin_spi_cs_1 -1
```

```
set pin_spi_cs_2 -1
```

```
set pin_buzzer_invert 0
```

```
set pin_led_invert 0
```

Command: mixer

```
set mix_outputs 5
```

```
set mix_0 1 0 -100
```

set mix_1 1 1 -100

set mix_2 1 2 100

set mix_3 1 3 100

set mix_4 2 0 100

set mix_5 2 1 -100

set mix_6 2 2 100

set mix_7 2 3 -100

set mix_8 3 0 -100

set mix_9 3 1 100

set mix_10 3 2 100

set mix_11 3 3 -100

set mix_12 4 0 100

set mix_13 4 1 100

```
set mix_14 4 2 100
```

```
set mix_15 4 3 100
```

```
set mix_16 11 4 100
```

```
set mix_17 0 0 0
```

Command: get output_

```
set mixer_output_limit 100
```

```
set output_motor_protocol PWM
```

```
set output_motor_async 1
```

```
set output_motor_rate 480
```

```
set output_motor_poles 12
```

```
set output_servo_rate 50
```

```
set output_min_command 1000
```

```
set output_min_throttle 1070
```

```
set output_max_throttle 2000
```

```
set output_dshot_idle 550
```

```
set output_dshot_telemetry 0
```

```
set output_0 M N 1000 1500 2000
```

```
set output_1 M N 1000 1500 2000
```

```
set output_2 M N 1000 1500 2000
```

```
set output_3 M N 1000 1500 2000
```

```
set output_4 S N 1000 1500 2000
```

```
set output_5 M N 1000 1500 2000
```

```
set output_6 M N 1000 1500 2000
```

```
set output_7 M N 1000 1500 2000
```

```
set pin_output_0 25
```



```
set pin_output_1 26
```

```
set pin_output_2 14
```

```
set pin_output_3 27
```

```
set pin_output_4 12
```

```
set pin_output_5 -1
```

```
set pin_output_6 -1
```

```
set pin_output_7 -1
```

The last two tables refer to the mixer and motor settings. I've also included all of these tables in the project files - so you can save them for later reference when copy-pasting into the CLI.

After pasting the table for *get pin*, make sure the buzzer is not inverted to shut it up (set pin_buzzer_invert 0), making it beep only when it's supposed to.

Then, we need to copy and paste the *mixer* settings to open up a channel for the servo - change the number 11 to any other available input channel you desire to use on your controller, channel 11 on mine correlates to the left slider.

The command *get output_* with the underscore shows the default motor settings - what I did was copy a motor line and change the zero-based output number to 4 for the 5th motor being the servo and put an *S* for servo instead of *M* for motor. The line *set output_4 S N 1000 1500 2000* assigns a range for the servo.

Once that's set, we'll type *save*, then *reboot*. Disconnect and reconnect to continue configuring.

In the Ports tab, set UART 2 to GPS under Sensor Input, make sure MSP is turned off, and select the correct GPS baud rate.

On UART 3, disable Serial RX since we're not using an external receiver, and enable MSP for communication with the minimOSD.

Click on save and reboot, reconnect, and you should see the GPS icon appear at the top right. If you do not, then you may need to swap UARTs

for each feature.

In the *Configuration* tab, the GPS feature must also be enabled. On the right, select when you want the buzzer to sound, such as when you arm, when the battery is low, when you have gained 8 satellites, and so on. At the top, give your craft a name and enable the barometer and magnetometer if you plan on using those sensors. Under *Board Sensor and Alignment*, set the pitch degrees to 90, gyro and mag alignment to clockwise 180 degrees - this ensures the IMU is perceived in the correct orientation. Hit save and reboot.

In the *Setup*, you can calibrate the Accelerometer and Magnetometer if needed.

In the *Power and Battery* tab, set On Board ADC as the meter source, a number around 200 as the scale and 5 as the divider value - the scale can be adjusted plus minus 10 points for further accuracy. When you hit save, the battery voltage should appear at the top.

In the *Receiver* tab, set SPI RX as the receiver mode since we're using the ESP-NOW communication protocol and select the standard Receiver

Provider. Check that your channel mapping on these 8 channels is correct according to your controller's inputs, then hit save and reboot.

There we go! It's receiving input from my Radiomaster TX12 controller that is on ESP-NOW with an ESP32S3 in its JR bay as the TX link communicating with the flight controller.

For the transmitter setup and complete firmware installation instructions, read my previous guide: <https://www.elektormagazine.com/labs/fly-your-esp32-drone-on-betaflight-with-radio-controller-esp-fc-guide>

Check that the four control stick channels, the switches, and the scroller wheel work as intended.

Head to the *Modes* tab, add a range for *Arm* and flick your desired toggle switch into position, then repeat this with another switch for activating the *Buzzer*, and permanently enable *Airmode* for smoother flight when arming by selecting the same AUX channel as on *Arm*. Finish here by clicking save.

Then there's the *GPS* tab, this is more for checking that your GPS unit is working: you'll see the number of satellites and their link quality. At least 5 locked satellites gives you a 3D fix and a minimum of 8 is required to arm the drone.

Let's head to the *Motors* tab, where we set the *Mixer* to Custom, *ESC Protocol* to PWM, and the highest *Frequency* to 480hz. To test the motors and servo from the configurator, we need to enable an option accepting the risk, and by dragging the sliders on the right, we're able to control each motor individually or all together.

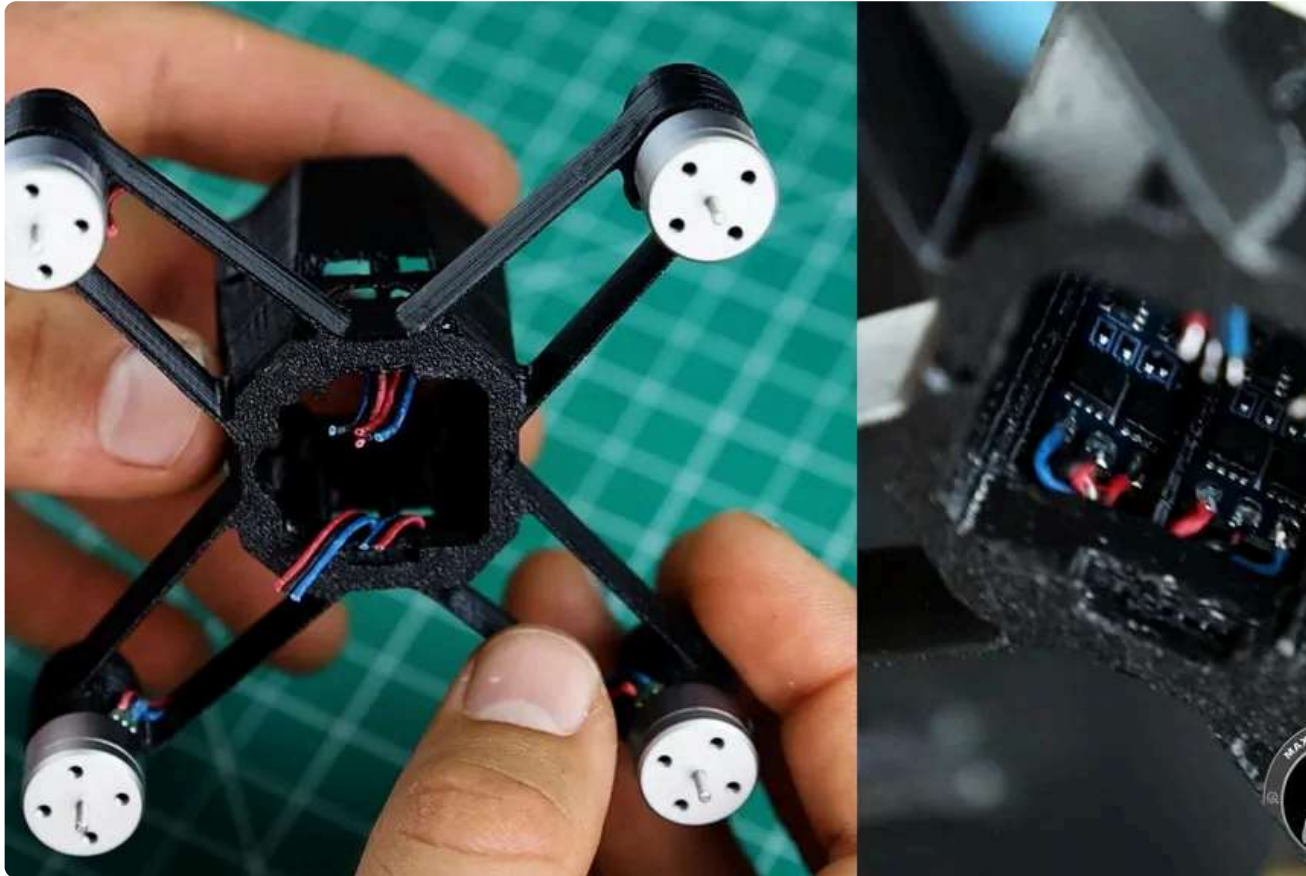
With everything programmed, not only can we now see the FPV camera feed with the OSD overlay in action - once we take the setup outside and the GPS locks onto at least eight satellites, we can arm the quad, spin up all four motors, control the micro linear servo, and beep the buzzer all from the controller.

To get the ESC LEDs blinking along with the buzzer's beeps, we'll bridge the LED and buzzer negative (BZ-) pads together. This ensures the LEDs conduct via the MOSFET along with the buzzer.

To give you an idea of possible upgrades - the back of the flight controller has pads for a current sensor module, as well as three extra servo or ESC outputs if you want to expand to more motors.

Since we cannot simply shove the whole circuit into the frame as-is, we need to desolder every motor and module, dismantling them back to their previous, pre-wired state.

Step 11: Installing Motors and ESCs



For the main assembly, we'll begin by mounting the 1104 mini brushless motors using the supplied screws.

I chose a pusher setup for the design, installing the motors upside-down. From what I've learned, positioning the arms above the propellers can help improve airflow efficiency, giving the drone slightly better performance for the same power.

Another cool detail is the 3D-printed motor cones that snap on with a tight fit, which not only help smooth out airflow but also neatly cover the screw holes.

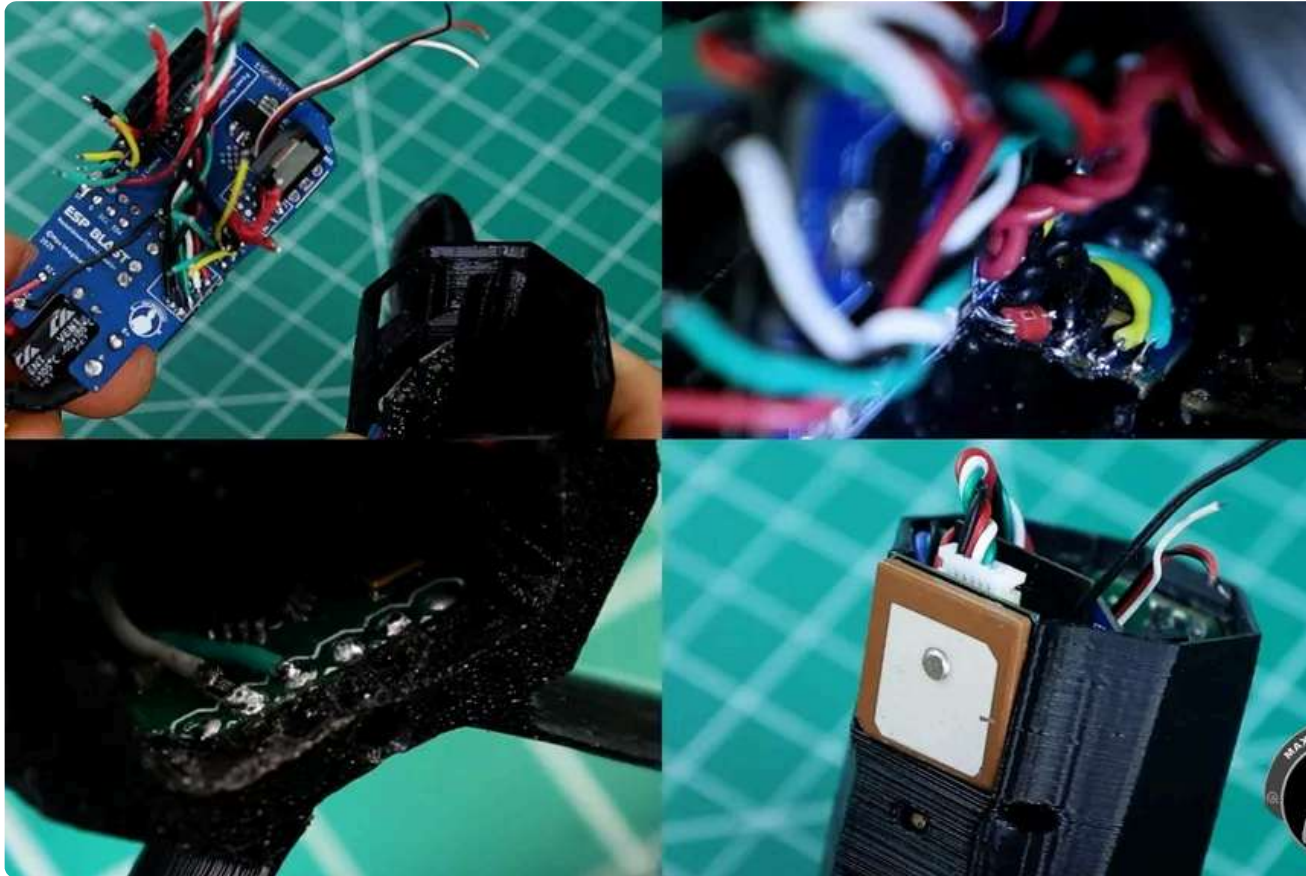
Now, we'll trim the motor wires so they don't separate as we shove them through the arm channels to the ESC bay inside. Using a pair of needle-nose pliers and tweezers make the job a lot easier.

While wearing an anti-static wrist strap to prevent your body's electrostatic discharge from damaging the micro ESCs, we'll slide them into their holders within the frame that prevent them from moving around without any glue. Push them in until their LEDs appear centered facing out through the openings.

From the bottom, we'll trim, strip, and tin the ends of the wires so they can get soldered onto their respective ESCs. For each motor pair on a wall, the

polarity is mirrored, with the black wire always soldered to the center pad.
Repeat the same wiring on the other side.

Step 12: Installing & Wiring Frame Components



To prepare the flight controller for its way into the frame, let's trim protruding solder joints flat and cut the XT30 connector wires short based

on how it fits with the tail cone on and solder it to the battery pads with the red wire wrapping around the capacitor.

However, before inserting the flight controller, we need to rearrange the wires on the barometer so that the IMU doesn't scrape those pads on its way in. This module snaps into its cavity first so it is out of the way and its wires must swerve around so they don't get caught.

Another thing not to miss, is to do some prewiring for the minimOSD and the GPS with four of its wires.

We'll even relocate the ESC wires to the flight controller as it's less awkward soldering this way. The outer ESC wires should be half the length of the inner ones as they go to the ESCs closest to the rear of the drone.

We'll tin the wire ends and insert the board into the frame - and yes, it may take a bit of force, but not too much to get the flight controller all the way in until everything is perfectly aligned.

Here comes the most fun part of this whole assembly (hint: not): Soldering wires inside this tube of a frame.

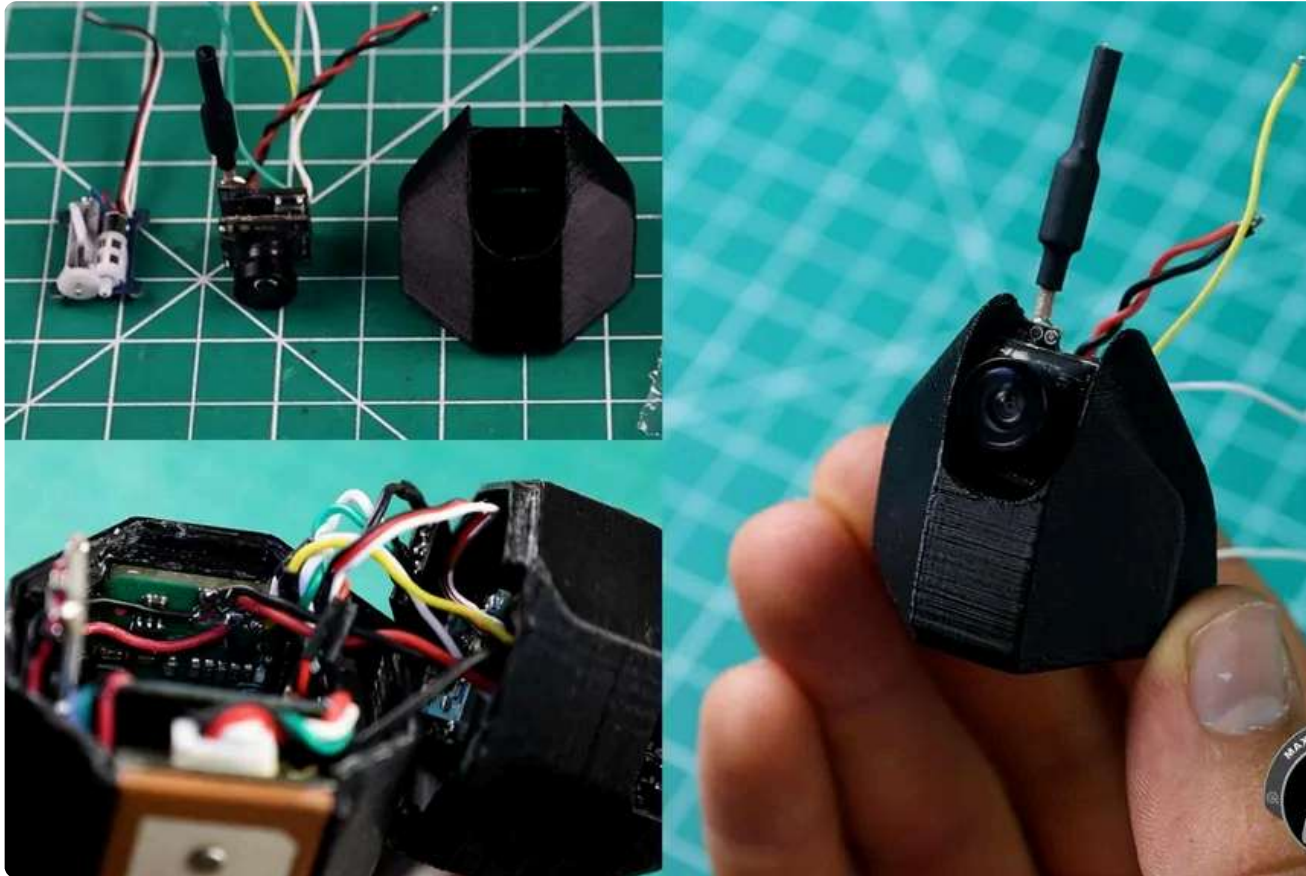
The ESC wiring may require shorter wires at certain points, but in any case - all of them must lay flat to stay out of the way of the battery.

We'll also connect the barometer by routing its wires over the shoulder of the flight controller and estimate the right lengths for each to get routed. Then, we'll prepare the minimOSD module by plucking its pin headers out in order for it to fit the frame. It slides in nice and easy, getting soldered with wires to its RX and TX pads - the latter via a 10K 0805 resistor.

At the camera side of the minimOSD, we'll solder wires for 5V and GND from the buck converter.

Then, we'll connect the GPS - which for now, can hang loosely. We'll test the battery pads against short circuits before doing a check-point power up to see that every module works - using a lab bench power supply with a smoke stopper in series (if possible) is safest.

Step 13: Making Tilting FPV Camera Nose



Great. Now we can shift focus to the nose cone containing the tilting camera assembly with the servo, 1.5mm steel wire, and some M1 screws.

Since we won't need Betaflight camera control, we can desolder the green SmartAudio wire and leave the other two connected.

We'll widen the screw holes, mount the micro linear servo using four shortened M1 screws, then slide on the 3D-printed servo fork that drives the camera, and trim the excess plastic from the servo horn.

Next, let's assemble the pivoting camera mount. Insert a 2cm piece of 1.5mm metal wire as the pin it's moved by, slide it into the forks, and snap each side of the mount into the ball sockets so the camera can tilt when driven by the servo.

Now, we can slide in the micro All-In-One 5.8GHz FPV camera with VTX and secure it with some drops of superglue.

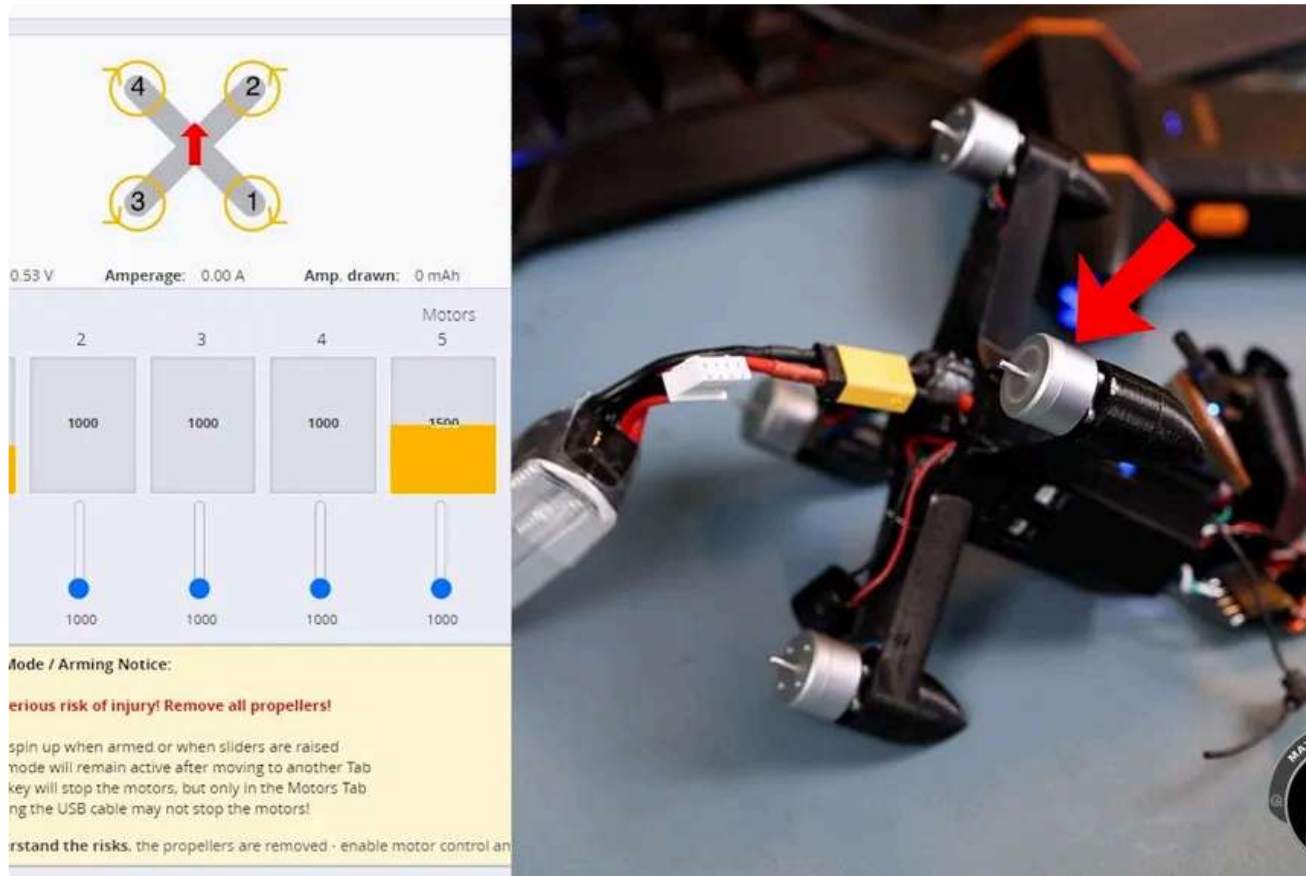
We'll solder the servo wires to the pre-wired ones on the flight controller and use heat-shrink tubes to wrap the exposed parts. Then, solder the camera's video out, video in, power, and ground wires to the minimOSD.

Let's power up the quad and confirm to see that we're getting OSD overlay on the camera feed.

Knowing the whole circuit works, let's anchor down any loose wires to the walls with superglue so they're well out of the way for the battery to slide in.

To program the flight controller within the frame, we use a custom wired 5-pin male to female header adapter that plugs right into the same FTDI port - allowing you to connect the drone to your computer and access it from Betaflight for all configurations like before.

Step 14: Testing Motors



Let's spin up motors 1 through 4 to confirm they're mapped in the correct order, as shown in the reference at the top left. Then verify that each motor spins in the correct direction to generate lift.

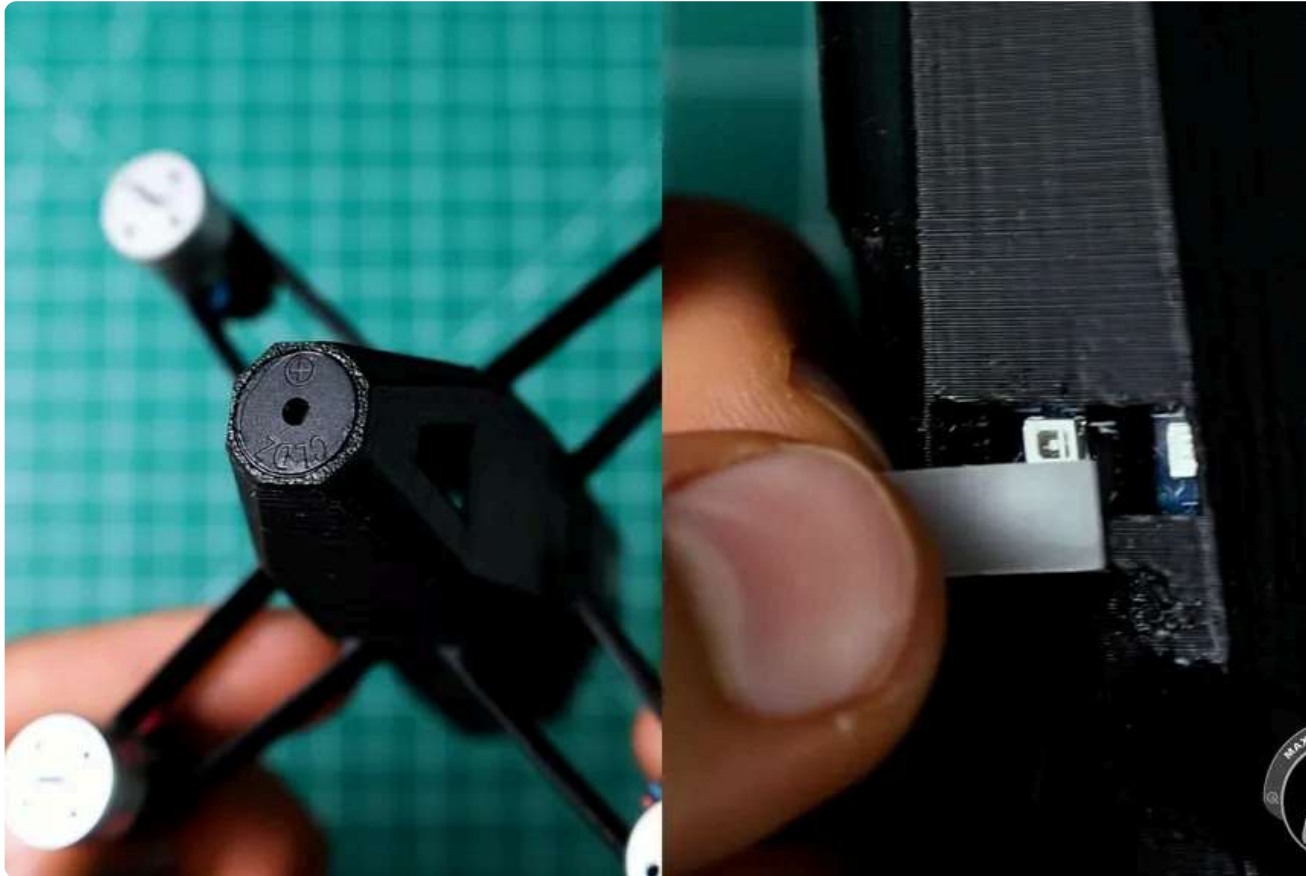
If one isn't, swap the motor polarity at the ESC to reverse it as software polarity reversal is not supported.

If you miss any of these steps, you may end up with a quad that behaves like it wants to impress you with backflips or worse, turn into a lawnmower.

This was the stage of the project where I realised a pair of motors had mismatched pinouts, resulting in it flipping. So, I just had to swap the pins on outputs 2 and 3, and then it could fly. Fortunately, you do not need to go through this process if you are building the same drone, using my preset CLI tables.

This is also a good time to check that the camera gets the servo's full range of motion for tilting.

Step 15: Tail Cone and LED Diffusers



Let's unplug the quad and continue securing parts with some superglue like the GPS, any other loose wires, and the minimOSD, then trim a corner of the GPS' connector so the ESP32's antenna fits when closing the top.

Now we can conveniently tuck away the programming header without unplugging it and close things off at the top with the nose cone.

With the wiring complete and everything running properly, we can focus on the more physical attributes of the quad like the tail cone which conceals the battery and contains the buzzer which sits superglued at the end.

To diffuse the LEDs, we'll sand a piece of translucent plastic sheet down to half a millimeter, get the measurements of an opening and cut out two identical 15x6mm panels that get superglued in these openings.

Before flying, it might be smart to drop a bit of superglue around the edges of the nose cone to prevent it from falling out of the frame.

Step 16: Preparing for Flight



Now let's mount the props; I'm using a set of GEMFAN 2540 tri-blade, 2.5-inch polycarbonate propellers. If these do not hold a tight fit, you may need

to lightly sand the motor shafts and the mating surface inside the prop hub to improve grip before gluing them in place.

The receiver antenna can clip into this mount at the back of the nose cone to prevent it from moving around.

Now the drone is complete, weighing in at 100 grams without and 136 grams with the battery.

It's pretty convenient that you can swap batteries in between each flight and connect the drained ones with your balance charger. I recommend charging these 450mAh batteries at 0.4A and no higher than 0.5A.

When powering on the drone, bundle up the wires and connectors so that the tail cone can fit over and clip into place.

If you power off the transmitter or lose signal with the drone in flight, the drone will begin to beep and flash its LEDs. Flip the designated toggle switch to get the buzzer beeping manually in case you lose it in some tall grass.

At the rear, the red LED indicates it's powered on and the blue status LED blinks 3 times when the drone is not ready, it blinks once when it has

enough satellites for you to arm, and stays solid when in flight. The arm switch starts up the propellers at idle speed before taking off.

My Fatshark Scout FPV headset is automatically bound to the All-In-One cam's VTX on its channel through an auto search on Band A. The pointed button at the top of the headset allows me to record the video feed as I'm flying so I can show it to you.

I felt like the drone could do with a platform that can be used as a launching pad and display stand, so I quickly 3D designed and printed one out of PLA which will come in handy in the field. The STL file for this stand comes included with your purchase of the 3D design files.

Step 17: Tuning PIDs



Now, let's turn off GPS so we can test the drone indoors and work on tuning it. Here is the process I went through:

Starting with the rates, I needed to gradually raise the sensitivity and rate values on pitch and roll to turn this line into more of a curve which made the drone more responsive when moving around.

Since its yaw was unresponsive, I had to do the same on its sensitivity and rate values which I prefer to keep lower than on pitch and roll so it does not spin out of control. Turning began to feel a lot more normal.

Now, for the PIDs. For best results, I went outside so I could push it a bit more. It seemed to hover around perfectly fine, but I needed to go out to a field to see how it behaves under some higher speeds.

Not only did I feel some yaw drift - when coming from higher speeds to a slow down, it wobbled heavily on the roll axis.

While tuning the PIDs and testing it some more in my yard, I happened to crash the quad and break the only 2.5" props I had. Being low on spares, I had to use what I had available, which was a set of 4-bladed 2" 2020 propellers. They may be smaller, but they do the job. The nice thing about these is they offer a really tight fit without any superglue like any non-screw mounted props should, making them easy to replace.

I continued with the PIDs. From the default 5" drone settings, I raised the P's, lowered the I's, and lifted up the D's, which although produced better results, it still wobbled from side to side when doing throttle punch outs which is a great way to observe wobble, so on roll, I raised the I value and even the dampening to reduce this wobble effect, resulting in a more well-managed descent from shooting it up.

For a simplified set of PID tuning guidelines, I found this cheat sheet on Reddit really helpful:

https://www.reddit.com/r/fpv/comments/14qbl2y/very_very_very_handy_pid_tuning_cheet_sheet_i/

Since it still needed more tuning to stop the nose from wandering at higher throttles and yaw from drifting, I called in an expert for guidance - who is Ben Biggs from Australia - the guy consistently breaking high-speed drone world records with his much larger rocket drone - The Blackbird. By the way, you can follow Ben Biggs on Instagram and YouTube to keep up with his record-breaking drone journey.

I told him what I was building, shared with him my settings, and described the symptoms. He suggested a few important PID changes - increasing the

I term on yaw to fix the drift and adding more damping (D) to reduce the wobble - which actually helped shape the **PIDs** I landed on: (Numbers in the order of P-I-D)

ROLL: 80-65-50

PITCH: 80-60-46

YAW: 35-110-0

Rates: (Center Sensitivity, Max Rate, & Expo)

ROLL: 650-750-0.00

PITCH: 650-750-0.00

YAW: 350-250-0.00

TPA: 0.10

TPA Breakpoint: 1100

The values I present may differ for your drone depending on its size, the sensitivity/responsiveness you're comfortable with, and improvements you may see are necessary to get it rock steady at higher speeds.

Step 18: Speed Runs



Now, it's time for some epic speed runs - the sound of it passing at high speed makes me ecstatic.

I tested it at a long stretch of country road for the extra space to see what it can really do - speeds may not be record-breaking, but the results were satisfactory for a small high-speed drone with non-ideal 2" props.

Currently, it seems to fly up to 100 kilometers per hour without a problem, but anything over that still makes it wobble and drift on the yaw axis on its current PIDs, requiring a few more focused days of tuning.

We could keep refining it on and on, but high-speed drone refinement is certainly a journey and not a destination.

I'm genuinely happy with the progress so far and am confident that with higher-pitched 2.5" props (linked in the BOM), a smooth-sand finish on the 3D printed body, GPS lock done prior to flight using a weaker battery, and fully-charged cells at 4.35V (on the Lithium-High Volt (LiHV) battery that was used), this thing will fly exactly how I want it to.

I may share my improved PID profile with updates on how it flies in a future Instagram Reel/YouTube Short.

The fun part is that you have the opportunity to take this project further and see even better results than on my drone!

Step 19: Conclusion

Build The Smallest ESP32 Brushless Rocket Drone | ESP-BLAST

Max Imagination



Watch on



Again: the files and bill of materials are linked at the start of this Instructable; order the parts, start 3D printing, and before you know it, you'll be launching your own mini rocket into the sky.

Can't wait to hear you "blast off" with your own ESP-BLAST high-speed quad, breaking my speed record!

Watch the full, detailed video tutorial here: <https://youtu.be/pUi1T12QYAU>